AI-Resume Parser Documentation

A comprehensive AI-powered resume parsing system that converts resumes from various formats (PDF, DOCX, images, text) into structured JSON data using Azure OpenAI's GPT models. The system provides both a FastAPI backend for programmatic access and a Streamlit web interface for easy file uploads and result visualization.

1. Project Description

The AI Resume Parser is an intelligent application designed to convert resume files in multiple formats (PDF, DOCX, Images, and Text) into structured, machine-readable JSON data. The application solves the critical problem of resume data standardization and extraction, leveraging Azure OpenAI Models (OCR capabilities) and AI-powered data parsing to automate resume screening and analysis workflows. This privacy-first system stores all files in-memory without any disk footprint, making it ideal for secure, confidential resume processing.

2. Objectives

- Provide automated resume parsing from multiple file formats with high accuracy and consistency
- Enable fast, scalable batch processing of multiple resumes with per-file error tracking and status reporting
- Deliver a user-friendly web interface for single and bulk resume uploads with intuitive tabbed data visualization
- Implement a privacy-focused architecture that keeps all files in-memory without persisting to disk
- Support seamless file format conversion and preprocessing to ensure uniform data extraction quality
- Generate structured, validated JSON output compatible with downstream recruitment and HR systems

3. Features

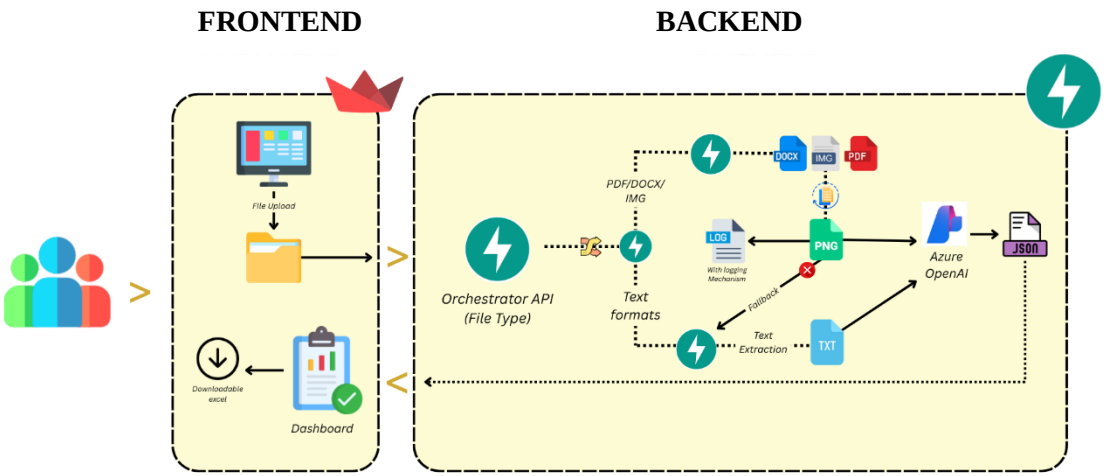
- **Multi-Format Support** - Accepts PDF, DOCX, DOC, JPG, PNG, BMP, TIFF, JFIF images and text-based files (TXT, MD, JSON, CSV, XML, YAML, code files)
- **Intelligent File Routing** - Automatically routes files to image-based or text-based parsing pipelines based on file extension
- **Batch Processing** - Process multiple resumes simultaneously from ZIP archives with parallel execution (max 2 workers) and per-file tracking
- **Azure OpenAI Integration** - Advanced OCR extraction using Azure's industry-leading OpenAI capabilities
- **AI-Powered Extraction** - Structured data extraction using Claude or Azure AI models with intelligent field mapping
- **In-Memory File Storage** - Privacy-first architecture storing all files temporarily in Streamlit session state with automatic cleanup
- **Inline File Preview** - View original resume files (PDF, images, text, Word documents) directly in the application without external downloads
- **Tabbed Data Visualization** - Seven organized tabs displaying Bio, Description, Skills, Languages, Employment History, Qualifications, and Original Resume
- **Real-Time Processing Metrics** - Track processing time, success/failure rates, and detailed error messages for debugging
- **Automatic Session Cleanup** - Secure automatic cleanup of all stored data when browser closes or session expires

4. Technology Stack

- **Programming Language**: Python 3.9+
- **Framework / UI**:
  - Backend: FastAPI (Port 8000) with Uvicorn ASGI server
  - Frontend: Streamlit (Port 8501) with Session State management
- **AI / API Service**:
  - Pydantic for data validation and schema enforcement
- **Cloud Platform**: local
- **Other Tools**:
  - `python-docx` - Word document parsing and extraction
  - `pdf2image` / `PyPDF2` - PDF conversion to images and text extraction

- Pillow - Image processing and display
- uvicorn - ASGI server for FastAPI
- requests - HTTP client for API calls
- python-multipart - File upload handling
- aiofiles - Async file operations

5. Workflow



5.1. System Architecture

The Resume Parser follows a **client-server architecture** with clear separation of concerns:

```
┌─────────────────────────────────────────────────────────┐
│                    STREAMLIT FRONTEND                   │
│  File Upload → Parse Button → Display Results → Download │
└─────────────────────────────────────────────────────────┘
                          │
                   HTTP POST /parse
                          ↓
```

```
┌──────────────────────────────────────────────────────────┐
│                      FASTAPI BACKEND                     │
│                                                          │
│  1. Orchestrator: Route by file type                     │
│  2. Converter: PDF/DOCX/IMG → PNG/Text                   │
│  3. Vision API: OCR extraction (Azure)                   │
│  4. AI Parser: Structured data extraction                │
│  5. Validation: Pydantic models check                    │
│  6. Response: JSON with file_id                          │
│                                                          │
│  Services:                                               │
│  - azure_vision.py   → OpenAI API calls                  │
│  - converter.py      → File format conversion            │
│  - logger.py         → Processing logs                   │
│  - schemas.py        → Data validation                   │
└──────────────────────────────────────────────────────────┘
                              |
                   HTTP Response JSON
                              ↓
┌──────────────────────────────────────────────────────────┐
│                 Session State (In-Memory Storage)        │
│                                                          │
│  - parsed_data (ResumeData)                              │
│  - file_bytes (Binary)                                   │
│  - original_filename (String)                            │
│  - file_id (UUID)                                        │
│                                                          │
│  Auto-cleared when session ends or browser closes        │
└──────────────────────────────────────────────────────────┘
```

Data Flow:

User → UI → Backend Logic (File Routing & Processing) → Azure OpenAI → AI Parser → Validation → Response → Frontend Display → Session Storage

6. Prerequisites

- Python 3.9 or higher
- Microsoft Azure account with OpenAI API enabled
- Azure OpenAI API key and endpoint URL
- 4GB RAM minimum for processing large PDFs
- Modern web browser (Chrome, Firefox, Safari, Edge)
- Virtual environment management tool (`venv` or `conda`)
- Internet connection for Azure API calls
- Optional: ZIP file creation tool for batch uploads

7. Installation & Setup

7.1 Clone the Repository

- ➢ git clone https://github.com/yourusername/AI-resume-parser.git
- ➢ cd AI-resume-parser

7.2 Create Virtual Environment

On Windows

- ➢ python -m venv venv
- ➢ venv\Scripts\activate

On macOS/Linux

- ➢ python -m venv venv
- ➢ source venv/bin/activate

7.3 Install Dependencies

- ➢ pip install -r requirements.txt

Key dependencies:

- fastapi==0.104.1
- uvicorn==0.24.0

- streamlit==1.28.0
- python-docx==0.8.11
- Pillow==10.0.0
- azure-cognitiveservices-vision-computervision==0.12.0
- python-multipart==0.0.6
- requests==2.31.0
- pydantic==2.4.2

8. Configuration

8.1 Environment Variables

- Create a .env file in the project root with the following variables:

8.2 Configuration

- AZURE_OPENAI_KEY=your-azure-openai-key-here
- AZURE_OPENAI_ENDPOINT=https://your-resource.openai.azure.com/

8.3 Application Configuration

- API_BASE_URL=http://localhost:8000
- FRONTEND_URL=http://localhost:8501
- MAX_FILE_SIZE_MB=25

8.4 Backend Configuration (config/settings.py)

- ALLOWED_FORMATS = {
  '.pdf', '.docx', '.doc', '.jpg', '.jpeg', '.png', '.bmp','.tiff', '.tif', '.jfif', '.txt', '.rtf', '.json',
  '.xml', '.yaml', '.yml', '.py', '.js','.html', '.css', '.java', '.sh', '.odt', '.tex',
  '.rst','.log','.md','.csv'
  }
- MAX_FILE_SIZE_MB = 25
- PARALLEL_WORKERS = 2 # For batch processing

9. Running the Application

- **Start Backend (Terminal 1)**

  Activate virtual environment

  venv\Scripts\activate **# Windows**

  (or)

  source venv/bin/activate **# macOS/Linux**

- **Run FastAPI server**
  - uvicorn main:app --reload
  - Backend available at : http://localhost:8000

- **Start Frontend (Terminal 2)**

  Activate virtual environment

  venv\Scripts\activate **# Windows**

  (or)

  source venv/bin/activate **# macOS/Linux**

- **Run Streamlit app**
  - streamlit run streamlit_app.py
  - Frontend available at : http://localhost:8501

Verify Services:

- Backend health check: curl http://localhost:8000/health
- Frontend: Open browser to http://localhost:8501

10. Usage Instructions

10.1 Single Resume Parsing:

Step 1: Open the Streamlit application at http://localhost:8501

Step 2: In the sidebar, select **"Single File Upload"** mode

Step 3: Click **"Choose a resume file"** and select your resume (PDF, DOCX, JPG, PNG, or text format)

Step 4: Click the **"Parse Resume"** button and wait for processing (2-30 seconds depending on file type)

Step 5: Navigate through the tabs to view parsed data:

- **Bio**: Name, email, phone, location
- **Description**: Professional summary and objective
- **Skills**: Technical and soft skills
- **Languages**: Languages spoken with proficiency levels
- **Employment History**: Work experience with companies, positions, dates
- **Qualifications**: Education, certifications, awards
- **Original Resume**: View the uploaded file inline

10.2 Batch Processing (Multiple Resumes)

Step 1: Select **"Batch Upload (ZIP)"** mode in the sidebar

Step 2: Create a ZIP file containing multiple resume files (all supported formats)

Step 3: Click **"Choose ZIP file"** and upload the archive

Step 4: Click **"Parse All Resumes"** button

Step 5: Monitor progress as files are processed in parallel (max 2 concurrent)

Step 6: View batch results showing:

- Total files count
- Successful processing count
- Failed processing count
- Per-file status (✓ success or ✗ failed with error)

Step 7: Click on individual resumes from the list to view parsed data and original file

10.3 Viewing Original Files

For PDF Files:

- Embedded PDF viewer displays in **Original Resume** tab
- Supports page navigation

For Images (JPG, PNG, BMP, GIF):

- Full-size image display with zoom capability

For Text Files (TXT, MD, JSON, CSV, XML, YAML):

- Text area with full content display
- Supports syntax highlighting

For Word Documents (DOCX):

- Extracted paragraphs and table content displayed
- Formatted for readability

11. Folder Structure

```
AI-resume-parser/
│
├── main.py  # FastAPI backend main application
├── streamlit_app.py # Streamlit frontend application
├── requirements.txt # Python dependencies
├── .env # Environment variables (create manually)
├── .env.example # Template for .env file
├── README.md  # User guide
├── WORKFLOW.md  # Technical workflow documentation
│
├── config/
│  ├── init.py
│  └── settings.py  # Application configuration
│
├── models/
│  ├── init.py
│  └── schemas.py  # Pydantic data models & validation
│
├── services/
│  ├── init.py
│  ├── azure_vision.py # Azure OpenAI API integration
│  ├── converter.py  # File format conversion logic
│  ├── parser.py  # Data extraction & parsing logic
│  └── logger.py  # Processing logs & metrics
│
├── utils/
│  ├── init.py
│  ├── ssl_config.py # SSL certificate configuration
│  └── validators.py  # File & data validation utilities
│
├── tests/
│  ├── test_converter.py # Unit tests for file conversion
```

```
│ ├── test_parser.py # Unit tests for parsing logic
│ └── test_api.py # API endpoint tests
│
└── sample_resumes/ # Sample files for testing
├── sample1.pdf
├── sample2.docx
└── sample3.txt
```

12. Error Handling

12.1 Backend Error Responses

400 Bad Request:

- Invalid file format or unsupported extension
- Empty file or corrupted file
- Missing required fields

413 Payload Too Large:

- File size exceeds 25MB limit
- Consider splitting large PDF files

500 Internal Server Error:

- Processing failed during conversion or parsing
- Azure API call failed
- Detailed error message included in response

12.2 Frontend Error Display

Single File Errors:

- Red error panel displays with icon and message
- Allows retry by uploading again

- File automatically discarded on failure

Batch Processing Errors:

- Per-file error tracking maintained
- Successful files continue processing
- Failed files show specific error message
- Overall batch status indicates success/failure ratio

Session State Errors:

- If data lost, message prompts re-upload
- Session automatically clears on browser refresh
- No persistent error state

12.3 Common Errors and Solutions

Error	Cause	Solution
Connection refused	Backend not running	Start FastAPI: `uvicorn main:app --reload`
401 Unauthorized	Azure credentials invalid	Verify API key and endpoint in .env file
File too large	Exceeds 25MB limit	Split large PDFs or use text format
Timeout	Large file processing	Increase timeout or use text-based files
Session lost	Refresh page or timeout	Re-upload resume for reprocessing
Unsupported format	File type not allowed	Use PDF, DOCX, JPG, PNG, or text formats

13. Security Considerations

13.1 API Key Management

- Store Azure credentials in `.env` file only
- Never commit `.env` file to version control

- Use `.gitignore` to exclude sensitive files
- Rotate API keys periodically
- Use environment variables in production

13.2 Access Control

- Implement authentication for production deployments
- Use HTTPS for all API communication (use SSL certificates)
- Validate file uploads server-side (never trust client input)
- Implement rate limiting on API endpoints
- Log all API access and parsing activities

13.3 Data Protection

- Files stored in-memory only (no disk persistence)
- Session state automatically cleared on timeout
- No database storage in default setup
- Implement HTTPS in production
- Consider GDPR compliance for resume data
- Never log personally identifiable information (PII)

13.4 Privacy Features

Privacy-First Design:

- Zero disk footprint - all files temporary
- Session state auto-clears on browser close
- No file retention after processing
- No third-party analytics
- Resume data only sent to Azure OpenAI

Considerations:

- Large files consume server memory
- Azure API may log processed content (check Azure privacy policy)
- Network transmission should use HTTPS in production

14. Limitations

- **File Size Limit**: Maximum 25MB per file; larger PDFs must be split before uploading
- **Session Storage**: In-memory storage limits concurrent users; not suitable for massive concurrent usage
- **Processing Speed**: Image-based parsing takes 10-30 seconds depending on file size and pages
- **OCR Accuracy**: Dependent on document quality; handwritten or low-quality scans may have extraction errors
- **Format Coverage**: Only supported formats are processed; unsupported formats show error
- **Batch Parallelization**: Limited to 2 concurrent workers; cannot be increased without memory impact
- **Database Support**: No built-in database; results not persisted after session ends
- **Scalability**: Single-server deployment; horizontal scaling requires architectural changes
- **Resume Standards**: Extraction quality varies based on resume format and layout consistency
- **Language Support**: Primarily optimized for English-language resumes